
Question 1. *Can you give examples of multivariate regression and time series tasks?*

Answer. Multi-output multivariate regression involves predicting multiple dependent variables (output quantities of interest) simultaneously based on multiple input features. Time-series tasks focus on analyzing and forecasting data points ordered in time (can be univariate or multivariate).

1. Weather Forecasting:

- Inputs: Historical weather data, atmospheric pressure, humidity, wind patterns
- Outputs: Temperature, precipitation, wind speed, cloud cover

2. Human Body Measurements:

- Inputs: Age, gender, height, weight, lifestyle factors
- Outputs: Body fat percentage, muscle mass, bone density

3. Vehicle Performance Prediction:

- Inputs: Engine specifications, vehicle weight, aerodynamics, tire type
- Outputs: Fuel efficiency, acceleration time, braking distance, top speed

4. Real Estate Valuation:

- Inputs: Property size, location, number of rooms, age of building, nearby amenities
- Outputs: Property value, rental price, estimated maintenance costs

5. Crop Yield and Quality Prediction:

- Inputs: Soil type, fertilizer usage, weather conditions, pest control measures
- Outputs: Crop yield, crop quality metrics (e.g., protein content, size, color)

6. Financial Portfolio Analysis:

- Inputs: Economic indicators, company performance metrics, market trends
- Outputs: Expected returns, risk measures, volatility for multiple assets

7. Manufacturing Process Optimization:

- Inputs: Raw material properties, machine settings, environmental conditions
- Outputs: Product quality metrics, production speed, energy consumption

8. Environmental Impact Assessment:

- Inputs: Industrial activities, population density, geographical features
- Outputs: Air quality indices, water pollution levels, biodiversity measures

9. Sports Performance Prediction:

- Inputs: Player statistics, team composition, opponent data, venue information
- Outputs: Predicted scores, player performance metrics, game duration

10. Medical Diagnosis and Treatment Planning:

- Inputs: Patient symptoms, test results, medical history, demographic information
- Outputs: Diagnosis probabilities for multiple conditions, recommended treatment options, predicted treatment outcomes

In all of the above examples, if a single model is used to predict all the quantities of interest at the same time, then that task is called as Multivariate Regression. A modeler can also choose to use different models for each quantity separately, then that becomes a univariate regression task.

The following are examples of time-series tasks.

1. Sales forecasting:

- Predicting future sales volumes based on historical sales data, considering seasonal patterns and trends.

2. Weather prediction:

- Forecasting temperature, precipitation, or other weather variables using historical weather data.

3. Energy demand forecasting:

- Predicting electricity or gas consumption patterns over time for a city or region.

4. Stock market trend analysis:

- Analyzing and predicting stock price movements over time, considering historical price data and trading volumes.

5. Website traffic prediction:

- Forecasting the number of visitors to a website based on historical traffic data, considering factors like day of the week and special events.

6. Economic indicator forecasting:

- Predicting future values of economic indicators like GDP, inflation rates, or unemployment rates based on historical data.

7. Pandemic spread modeling:

- Forecasting the spread of infectious diseases over time based on historical infection rates and other relevant factors.

□

Question 2. *Consider the business need of preparing Performance Improvement Plans (PIP) for employees. Break down into several sub-problems to get the full end-to-end solution.*

Answer. The comprehensive end-to-end business problem might include several sub-problems, some of which are AI problems and some are non-AI problems. Each AI sub-problem may involve either a predictive AI task or a Generative AI task. For instance, consider creating an HR performance improvement plan for a software developer. The ultimate deliverable is a document summarizing feedback, including the PIP's objectives, areas for development, methods for measuring improvement, and a two-quarter action plan. To produce this document, various sub-problems need to be addressed. One sub-problem is to summarize text from Manager feedback, 360 feedback, and self-assessment, which is known as text-to-text summarization, a GenAI task. Another sub-problem is to detect areas of improvement from the Manager's text, a text understanding task classified as a predictive AI task that can be approached as a text classification problem. The methods for assessing improvements could rely on a simple lookup table without needing an ML/AI model. Crafting the action plan involves a multi-modal input to text GenAI task. These four sub-problems must be solved to achieve the complete end-to-end solution. □

Question 3. *Could you please provide a few examples as to how this feature engineering can be done for a tabular data?*

Answer. Feature engineering is a crucial step in preparing tabular data for machine learning models. It can significantly improve model performance by creating more informative or relevant features. Here are some common feature engineering techniques for tabular data, along with examples:

1. Binning (Discretization): Convert age (continuous) into age groups (categorical).

```
def age_group(age):  
    if age < 18: return 'Minor'  
    elif age < 65: return 'Adult'  
    else: return 'Senior'  
  
df['age_group'] = df['age'].apply(age_group)
```

2. Interaction Features: Combine 'height' and 'weight' to create BMI.

```
df['bmi'] = df['weight'] / (df['height'] / 100) ** 2
```

3. Temporal Features: Extract day of week, month, etc. from a date column.

```
df['day_of_week'] = df['date'].dt.dayofweek
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)
```

4. Aggregation Features: Create features based on group statistics.

```
df['avg_price_by_category'] = df.groupby('category')['price'].transform('mean')
```

5. Encoding Categorical Variables: One-hot encoding for categorical features.

```
df = pd.get_dummies(df, columns=['color'], prefix='color')
```

6. Mathematical Transformations: Apply log transformation to skewed numeric features.

```
df['log_salary'] = np.log1p(df['salary'])
```

7. Text-based Features: Extract the length of the text from a string column.

```
df['description_length'] = df['description'].str.len()
```

8. Domain-specific Features: Create a 'is_holiday' feature for time series data.

```
import holidays
us_holidays = holidays.US()
df['is_holiday'] = df['date'].isin(us_holidays).astype(int)
```

9. Percentile Features: Create percentile ranks for numeric features.

```
df['salary_percentile'] = df['salary'].rank(pct=True)
```

10. Polynomial Features: Create polynomial terms for numeric features.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
poly_features = poly.fit_transform(df[['feature1', 'feature2']])
```

□

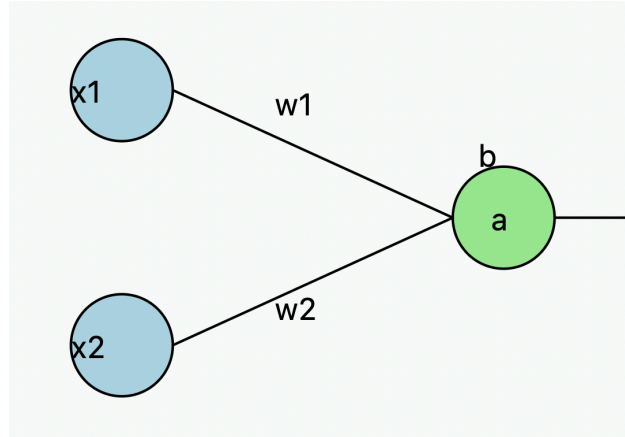


Figure 1: Caption

Question 4. *How are the weights and biases (parameters) of a neural network decided?*

Another Q: Are weight and bias defined by the problem solver?

Answer. The parameters of a neural network are learned from the data using the backpropagation algorithm. Training a neural network involves the following major steps

1. Start with an initial guess of all the weights and biases. E.g., LeCunn Initialization, Glorot Initialization, and He Initialization are some common strategies.
2. Feed a mini-batch of data through the neural network. Collect the output and store the intermediate results of each layer.
3. Compare the output of each data point in the mini-batch to the ground truth and compute the loss.
4. Calculate the gradient of the loss (derivative) with respect to each trainable parameter. Note that some parameters might be non-trainable (e.g., when using transfer learning). This process is called back-propagation.
5. Use the calculated gradients to update all trainable parameters. This completes one iteration.

A single iteration modifies all trainable parameters (weights and biases). Each iteration processes one mini-batch of data. When the count of iterations is enough to cover all data points at once, it marks the completion of one epoch. $\square$

Question 5. *Can you explain what Weights and Biases are with an example?*

Answer. Fig. 1 shows a simple neuron with two input features x_1, x_2 with weights w_1, w_2 and bias b . The output of this neuron is $a = f(w_1x_1 + w_2x_2 + b)$, where f is the activation function. f can be a linear activation or a nonlinear activation function. The most common nonlinear activation functions are sigmoid, ReLU (Rectified Linear Unit) or Softmax. $\square$

Question 6. *What is the difference between theta1 and w1 shown in the lecture slides?*

Answer. In the example shown in the class, w_1 was the vector that was used as weights to get the first neuron in layer1. w_1 had two components $[\theta_1, \theta_2]$. $\square$

Question 7. *Why do we need bias? What is the purpose of having bias?*

Answer. Bias in a neural network serves a crucial role in allowing the network to represent and learn patterns that do not necessarily pass through the origin. Here's why it's important:

- a) **Flexibility:** Bias allows the activation function to shift left or right, which is critical for the model to fit the data properly.
- b) **Representation power:** Without bias, the model would always be constrained to pass through the origin $(0, 0)$, which severely limits what it can represent.

$\square$

Question 8. *Does activation apply after each layer?*

Answer. Yes. A layer is made up of multiple neurons. An activation function is applied to each neuron. In PyTorch, layers are typically defined as a linear combination followed by an activation function. In contrast, in Keras and TensorFlow, the activation function is often included within the layer definition itself. Both methods yield the same result, with PyTorch making the two steps within a neuron more explicit. $\square$

Question 9. *Why is ReLU activation called a non-linear activation?*

Answer.

1. A function $f(x)$ is linear if it satisfies two properties:
 - Additivity: $f(x + y) = f(x) + f(y)$
 - Homogeneity: $f(ax) = af(x)$ for any scalar a
2. ReLU function: The ReLU function is defined as $f(x) = \max(0, x)$
3. Non-linear behavior: ReLU doesn't satisfy the linearity properties:
 - (a) It's not additive: Consider $f(1) + f(-1)$ vs $f(1 + (-1))$: $f(1) + f(-1) = 1 + 0 = 1$
 $f(1 + (-1)) = f(0) = 0$ $1 \neq 0$, so additivity doesn't hold
 - (b) It's not homogeneous: Consider $f(-2x)$ vs $-2f(x)$ when $x = 1$: $f(-2 * 1) = f(-2) = 0$
 $-2f(1) = -2 * 1 = -2$ $0 \neq -2$, so homogeneity doesn't hold
4. Graphical representation: If you plot the ReLU function, you'll see it's not a straight line. It has a 'bend' at $x = 0$, which is a clear indication of non-linearity.
5. Importance in neural networks: Non-linear activations like ReLU are crucial because they allow neural networks to learn complex, non-linear relationships in data. Without non-linear activations, a neural network, no matter how deep, would only be capable of learning linear transformations.

$\square$

Question 10. *Can you please explain the role of activation function?*

Answer. Non-linear activations like ReLU are crucial because they allow neural networks to learn complex, non-linear relationships in data. Without non-linear activations, a neural network, no matter how deep, would only be capable of learning linear transformations. $\square$

Question 11. *Can linear relation be used to explain ‘input increase , output reduce’?*

Answer. A function $f(x)$ is linear if it satisfies two properties:

- Additivity: $f(x + y) = f(x) + f(y)$
- Homogeneity: $f(ax) = af(x)$ for any scalar a

This includes situations where input increases, output reduces. Such a situation is called as negative correlation between input and output. $\square$

Question 12. *How do we decide which Activation function to apply?*

Answer. It can be decided on the basis of a careful cross-validation approach. Usually, ReLU activation (or their cousins) can be applied on the intermediate layers. On the output layer, sigmoid is used for binary classification, softmax for multi-class classification and linear for regression problems. $\square$

Question 13. *Is sum of a_1 , a_2 and a_3 equal to 1 in a softmax activation with three nodes for a 3-class classification problem?*

Answer. Yes, softmax activation ensures that the probability returned on the output layer all add up to 1. $\square$

Question 14. *As the number of layers increases, will the models have more parameters?*

Answer. Yes, the number of parameters increases with number of layers and number of neurons in a layer (for dense layers). $\square$

Question 15. *Is the analogy of ‘complex manifold to unfolded paper’ a mathematical result for neural networks?*

Answer. Yes, the transformation of the neural network data representation is a mathematical result. It comes from statistical learning theory. $\square$

Question 16. *How to decide on number of layers?*

Answer. The number of layers are decided by cross validation. $\square$

Question 17. *Does the input layer contain the same number of neurons as there are inputs?*

Answer. Yes. The input layer is a pass through layer with the same number of neurons as input signal. For example, in a table with 6 features, the input will contain 6 neurons. In an image with 28×28 pixels, the input layer will have 784 neurons. $\square$

Question 18. *Is vector DB (data base) required for NN generative task ?*

Answer. A vector database is a special database where elements (such as images, text, audio, video) are stored using their vector representations. Such a database makes retrieval efficient for retrieval augment generation tasks. $\square$

Question 19. *Where did we get 28×28 ? Can this flattening be done column-wise as well? Will the vector size not be $28 \times 28 \times 2$, 2 for black and white or will it be only 28×28 ?*

Answer. The size of the MNIST dataset is 28×28 . This choice has historical reasons.

1. Original data source: MNIST is derived from NIST's (National Institute of Standards and Technology) Special Database 3 and Special Database 1, which contained hand-written digits.
2. Preprocessing: The original NIST data had a resolution of 128×128 pixels, with the digits taking up most of this space.
3. Size reduction: The creators of MNIST (Yann LeCun, Corinna Cortes, and Christopher J.C. Burges) reduced the size to 28×28 for several reasons:
 - (a) Computational efficiency: Smaller images require less memory and processing power, which was especially important in the 1990s when MNIST was created.
 - (b) Focus on essential features: 28×28 was found to be sufficiently large to capture the essential features of digits while removing unnecessary background.
 - (c) Standardization: A consistent size makes it easier to use the dataset in various machine learning models without additional preprocessing.
4. Centering method: The digits were centered in the 28×28 field by computing the center of mass of the pixels, and translating the image so this point was at the center of the 28×28 field.
5. Anti-aliasing: When reducing from 128×128 to 28×28 , anti-aliasing techniques were used to maintain the quality of the digit shapes.
6. Practical considerations: $28 \times 28 = 784$ pixels, which is a manageable number of input features for many machine learning algorithms, especially at the time of MNIST's creation.

This size has become a standard in the field, allowing for easy comparisons between different models and approaches. Many newer datasets and challenges have used this size as a reference point, even when dealing with more complex images.

Grayscale images have only one channel, black being 0 and white being 1, and grayscale being a fraction. Sometimes Black is 0 and white is 255 with other colors being integers.

Color images have typically 3 channels, Red, Green and Blue. $\square$

Question 20. *How are the neurons decreasing every layer?*

Answer. The neural architecture is a design choice that has to be determined by cross validation. $\square$

Question 21. *Give an example of parameter calculation.*

Answer. Let there be a neural model with 784 neurons in the input layer (Layer 1), 100 neurons in first intermediate layer (Layer 2), 100 neurons in the next layer (Layer 3), 10 neurons in the next layer (Layer 4) and 10 neurons in the output layer (Layer 5).

Layer 1 does not have any learnable parameters.

Layer 2 has 100 neurons, each connected to 784 in the previous layer. Plus, all 100 neurons have a bias. So total number of parameters in Layer 2 is $784 \times 100 + 100 = 78500$.

Layer 3 has 100 neurons, each connected to 100 in the previous layer, Plus, all 100 neurons of Layer 3 have a bias. So, total number of parameters in Layer 3 is $100 \times 100 + 100 = 10100$.

Layer 4 has 10 neurons, each connected to 100 in the previous layer, Plus, all 10 neurons of Layer 3 have a bias. So, total number of parameters in Layer 3 is $10 \times 100 + 10 = 1010$.

Layer 5 has 10 neurons, each connected to 10 in the previous layer, Plus, all 10 neurons of Layer 3 have a bias. So, total number of parameters in Layer 3 is $10 \times 10 + 10 = 110$.

Total number of parameters is equal to $78500 + 10100 + 1010 + 110 = 89720$. $\square$

Question 22. *Do we use the final 10 class classifications in the output layer to classify the image?*

Answer. Yes, the output layer will have as many neuron as number of classes in the classification problem. $\square$

Question 23. *One single sample data set is 784 vectors, so for all such samples will we construct a single matrix as the $n \times n$ matrix as input?*

Answer. One data point has 784 pixels, which will appear as columns. One data point is a 784 dimensional vector. This is a row of the data set. If I have 10,000 images then my data set will contain 10,000 rows and 784 columns. Each column is a pixel, and each row is an image. $\square$

Question 24. *Are LLMs trained on labeled data since it is based on a neural network that calculates weight and bias from data?*

Answer. To train generative LLMs, datasets are created with input-output pairs. These are typically sentence-completion tasks. Part of a document will be given as input, and the rest will be given as output. The neural networks will be trained in a supervised manner using this dataset. This procedure is called as Language Modeling. $\square$

Question 25. *How do we quantify that the Neural Network is over trained?*

Answer. A neural network is overfitting to the training data if the training error is low and validation error is high. For classification problems, metrics such as F1 score, Recall and Precision are used. For regression, Root Mean Square Error is used. For language generation tasks, other metrics such as BLEU score, perplexity score are used to determine if a neural model has overfit. $\square$

Question 26. *Will every layer have the same weights and biases?*

Answer. No. Every dense layer will have neurons with its own set of weights and biases. Note that in a convolutional layer, neurons are arranged as channels. All neurons in a channel share the same set of weights and bias in a convolutional neural network. $\square$

Question 27. *In the example shown in class, does target prediction mean, predicting the number 4?*

Answer. Yes, in the example shown in the class, the input was an image of the number 4. The neural model must output the number 4. It does the task by predicting a probability over the classes 0,1,2,...,9. We want a trained model to give the highest probability for the class label 4. $\square$

Question 28. *How do we find ground truth?*

Answer. The ground truth is obtained by humans labelling the dataset. Usually obtaining the label for predictive tasks is expensive. There are label service providers who will employ humans to label data to train AI. $\square$

Question 29. *Is there any advantage of using linear regression using the neural network approach?*

Answer. Linear regression is a baseline model. Any neural network or gradient boosted tree model that we use must be significantly better than linear regression to justify the cost of developing such complex models. $\square$

Question 30. *Here, image is broken into 784 pixels and each pixel is compared with true target or the entire 784 pixel is compared with True Target ?*

Answer. Neither. The 784 pixels are the input. The output from the neural network is a probability over the class from the set (0,1,2,...,9). The ground truth is a class. $\square$

Question 31. *In our multiclass classification problem do we have a single loss function that fits all the 10 classes or do we have one loss function for each?*

Answer. Our loss function calculates the loss for each datapoint. The cross entropy loss considers all 10 classes to compute the loss for each data point. $\square$

Question 32. *Will the output / ground truth for each image will be a vector of 10 values, of which one of the number is 1 and all others 0?*

Answer. The representation where one entry in a vector is 1 (corresponding to the ground truth) and others being zero is called the one-hot vector. The ground truth will be a 1 hot vector. The output of the neural model will be a probability vector over the classes of the output. In our example, it will be a 10-dimensional probability vector. For a trained model, we want the probability of the true class to be maximum. $\square$

Question 33. *Is there any mechanism for these initial guesses for weights and biases?*

Answer. Yes, the initial guess is a random variable with a variance that is specified based on the number of neurons in that layer and number of neurons in the previous layer. Different researchers have provided slightly different formulae to calculate the variance to be used in the initialization. Later research has also shown that Batch Normalization (in convolution) and Layer Normalization (in multi head attention) are necessary in addition or instead of the correct initialization. This area of research is called as non-convex optimization methods for machine learning, a very active area of research. $\square$

Question 34. *If the input is 4, then the output is either 4 or something else. So the error is binary rt? Either it is predicted 4 or something else*

Answer. This approach of solving multiclass classification is called as OvR (One versus the Rest). Here 10 different binary classification problems are solved. Each problem is about, am I predicting class 0 or not, class 1 or not, etc. In a neural model, the multi class classification can be done directly using softmax layer, instead of an OvR approach. To complete this concept, OvO (One vs the others) is another approach of converting multi class problems into binary. In OvO, we consider several problems such as 1 vs 2, 1 vs 3, 2 vs 3 etc. Such an approach was necessary in the 1990s when Support Vector Machines which are fundamentally a binary classifier was the most popular model. Today, we do not need such approaches. $\square$

Question 35. *Can we reuse training and test data for different development and test iterations? Can we reuse test set for multiple iterations of testing?*

Answer. We can utilize training data and validation data multiple times. However, if we use test data to enhance our model, the test data essentially becomes validation data. This method is not advised because it eliminates having an unbiased assessment prior to deployment. $\square$

Question 36. *Is there a chance to predict the same probability for an output across classes?*

Answer. Theoretically, we can have a model output the exact same probability for multiple classes. Practically, this is rare. However, if a model is confused, then the probability of two classes could be close. For example, a binary classification problem with a model predicting 0.51 as the probability of the positive class. $\square$

Question 37. *Can we use same development data set until we reach less loss score?*

Answer. We typically keep the same development data for developing multiple different models until we get better evaluation metrics on the validation part of the development set. $\square$

Question 38. *Is it true that the optimizer updates both the weight and bias? Can you give one example to explain the two?*

Answer. Yes, the optimizer will update the weights and bias both in every iteration.

$$w_1^{(k+1)} = w_1^{(k)} - \eta \frac{\partial L}{\partial w_1} \quad (1)$$

$$b^{(k+1)} = b^{(k)} - \eta \frac{\partial L}{\partial b} \quad (2)$$

Here the superscript refers to the iteration count. All derivatives are evaluated at the current iteration count (k in this case). $\square$

Question 39. *Will the prediction set (y') look like $[0,0,0,0,0,0.1,0.1,0.8,0]$? Will this be the closest probability with ground truth?*

Answer. Yes, the prediction for a data point will look like $[0,0,0,0,0,0.1,0.1,0.8,0]$ if the ground truth is for the 8th index. $\square$

Question 40. *Should not we use neural network even for tabular data because feature engineering is done implicitly whereas in gradient boosted tree we would need to do it manually?*

Answer. Neural networks are very good when the features are correlated and there is a lot of contextual information to be gathered. They are bad in a tabular setting where the columns are different aspects of the data. We recommend to use Gradient Boosted Tree models for tabular Predictive AI tasks. $\square$

Question 41. *If i have tabular data say for customer credit history, can we not use neural network ?*

Answer. In such a case, it is recommended to use Gradient Boosted Tree model. You can also try a SARIMAX time series model. Rarely, will this problem statement need a neural solution. $\square$

Question 42. *If a model performs well on one set of test data, but it performs poorly on another set of test data. What do we conclude from this?*

Answer. In such a setting, please first check the distribution of the test data and the development data. If the two data distributions are different, then there is no surprise here. If the data distributions are same, then it indicates that your model is not generalizing to unseen data in the same distribution as training. This observation would suggest an overfit model. Try hyper parameter tuning. Further, look at expanding the data in terms of more data points or more features. $\square$

Question 43. *Does the data set of batch size 200 get picked randomly from 1000 samples or does this get fixed in start?*

Answer. Theoretically, we like it if every minibatch is picked randomly. However, due to computational data movement costs, we choose the first 200 in the data set. $\square$

Question 44. *Can batch size be changed across epochs?*

Answer. It can be changed, but typically kept the same. $\square$

Question 45. *Is there any commonly used value for learning rate, or is it on a case-by-case basis?*

Answer. The learning rate is a hyperparameter that is tuned on a case-by-case basis. Several practical tricks may be necessary to get training to work in complex situations such as training an LLM. Many companies guard these tricks as trade secrets that allow them a competitive advantage. $\square$

Question 46. *How much data should a data set have in a practical scenario?*

Answer. For training generative AI models, there are trillions of training data points. $\square$

Question 47. *For industry standards, is there a preference for using Tensorflow over Pytorch based on performance?*

Answer. The choice is based on learning curve of the team, code legacy reasons, and general comfort levels of the developer. TensorFlow had the advantage of a wider user community and embedded system deployment. PyTorch had the advantage of familiarity in research labs. Today both systems have converged. $\square$

Question 48. *Is it possible to use the CPU with PyTorch on our machine when we don't have a GPU available?*

Answer. Yes, PyTorch works with CPU, but training time is too much for practical use cases. $\square$

Question 49. *For people interested in working with models frequently and training them, would you recommend buying a GPU ? If yes, then which model should be good for general usecases. If no, then any suggestions for any cloud providers? I saw the rate cards for GPUs on cloud providers and found them typically cost prohibitive.*

Answer. For heavy duty use, an on-premise GPU is advisable. $\square$

Question 50. *Why does the algorithm move in the decent direction? is it because of the initial guess?*

Answer. The negative of the gradient is the steepest descent direction. From any point (including the initial guess), we move in the direction of steepest descent during optimization. $\square$